

between entities based on Transformers [46]. *In the first stage*, our framework avoids expensive all-to-all interaction modeling and extracts context features only *locally*. Specifically, we divide the scene into a set of local regions, where each local region is centered at one modeled agent. For each agent-centric local region, we extract context features from the local vectorized entities which contain rich information related to the central agent. *In the second stage*, to compensate for the restricted local receptive fields and capture long-range dependencies in the scene, we perform global message passing among agent-centric local regions by empowering the Transformer encoder with the geometric relationships between local reference frames. *Finally*, given the local and the global representations, a decoder produces future trajectories for all agents in a *single* forward pass. To further leverage the symmetries of the problem, we employ a scene representation that is agnostic about the translation of the global coordinate frame, in which we use relative positions to characterize all vectorized entities. Based on this scene representation, we introduce rotation-invariant cross-attention modules for spatial learning, which can learn local and global representations that are invariant to the rotation of the scene.

Our approach has the following clear advantages. *First*, by decomposing the problem into local context extraction and global interaction modeling, our approach can progressively aggregate information at different scales and model a large number of entities in the scene with high efficiency. *Second*, our method can learn representations robust to translation and rotation of the inputs via a translation-invariant scene representation and rotation-invariant spatial learning modules. *Third*, our model can make faster and more accurate predictions with much fewer parameters than the state-of-the-art approaches. We validate all the above advantages via extensive experiments on large-scale driving data. Our code will be publicly available.

2. Related Work

Traffic Scene Representation. Tackling the motion prediction problem requires learning rich representations from the elements of the traffic scene, including the high-definition map and the past trajectories of agents. Extensive works have used rasterized scenes as model inputs [4, 9, 12, 15, 19, 25, 43] and employed standard image models [11, 24, 26, 44] for learning. Specifically, these methods extract map elements (e.g., lane boundaries, crosswalks, traffic lights) from the high-definition map and use different colors or masks to render the scenes as bird’s eye view images. The past trajectories of agents are either rasterized as additional image channels [9, 12] or processed by temporal models like RNNs [2, 41, 42].

Rasterized methods are compatible with the mature techniques in computer vision but are also costly and inefficient

in learning. Recently, vectorized methods [17, 31, 49] have gained popularity for their efficient sparse encoding and the capability of capturing complex structural information. Unlike the rasterized approaches, these methods treat the scene as a set of entities associated with semantic and geometric attributes and learn the relationships between entities. VectorNet [17] models the interactions among lane and trajectory polylines with graph neural networks. It is also used as the backbone network by some following works [22, 32, 52]. LaneGCN [31] builds a lane graph from the lane segments and exploits multi-scale graph convolutional networks to learn representations for graph nodes. TPCN [49] extends point cloud models to learn from the spatial-temporal point set composed of trajectory waypoints and lane points. Our scene representation also falls into this category but differs in that all vectorized entities are characterized by relative positions, making our representation *translation-invariant*.

Motion Prediction. Since social interactions are ubiquitous in traffic scenarios and significantly impact the future motions of traffic agents, many motion prediction approaches have considered the dependencies between agents’ behaviors and reasoned agent-agent interactions using social pooling [2, 13, 23], graph neural networks [8, 27, 31, 37], or attention mechanisms [17, 30, 32, 36, 38, 43, 48, 50]. Inspired by the success of Transformer models [46] in a wide range of domains [5, 7, 14, 16], some recent works employ Transformers in the motion prediction task to model spatial relationships, temporal dependencies, and relationships between agents and map elements [21, 30, 32, 36, 38, 50, 51]. In comparison, our Transformer architecture differs from existing ones by learning local and global representations hierarchically. This hierarchical strategy helps the model learn multi-scale features and is more efficient than those performing all-to-all message passing along the space and the time axes. Moreover, we model multiple agents via an agent-centric representation that is invariant to translation and rotation of the scene. The hierarchical architecture and the symmetric designs enable our method to achieve the state-of-the-art prediction performance with fewer parameters and less computational cost than other approaches.

3. Approach

3.1. Overall Framework

An overview of our proposed framework is illustrated in Fig. 1. We first organize a traffic scene as a collection of vectorized entities. Based on this scene representation, our framework hierarchically aggregates spatial-temporal information in the scene. In the first stage, we encode rotation-invariant local context features for each agent. The aggregation of ego-motion, neighboring agents’ motions, and local map structure can provide rich information related to the

central agent. For each local region, we aggregate the local information into a single feature vector by sequentially modeling agent-agent interactions per time step, temporal dependencies per agent, and agent-lane interactions at the current time step. After aggregation, the feature vector contains rich information related to the central agent. On the other hand, the computational complexity is reduced from $O((NT + L)^2)$ to $O(NT^2 + TN^2 + NL)$ by the factorization of the space and the time dimensions and is further reduced to $O(NT^2 + TNk + N\ell)$ by limiting the radius of the local regions, where $k < N$ and $\ell < L$.

While the local encoder can learn rich representations locally, the amount of information is limited by the range of the local regions. To avoid sacrificing prediction performance, we further employ a global interaction module to compensate for the restricted local receptive fields and capture scene-level dynamics, in which we perform message passing among local regions. The global interaction module can significantly enhance the expressiveness of the model at the cost of $O(N^2)$ complexity, which is relatively lightweight compared with the local encoder.

The multi-agent motion prediction problem exhibits translation and rotation symmetries. Existing methods [17, 31] re-normalize all vectorized entities w.r.t. each agent and make single-agent predictions multiple times to achieve invariance. This paradigm scales linearly w.r.t. the number of agents. By comparison, our model can make predictions for all agents in a single forward pass without sacrificing invariance via using the translation-invariant scene representation and rotation-invariant spatial learning modules. We illustrate the model components in more detail as follows.

3.3.1 Local Encoder

Agent-Agent Interaction. The agent-agent interaction module aims to learn the relationships between the central agent and the neighboring agents for each local region at each time step. To exploit the symmetries of the problem, we introduce a rotation-invariant cross-attention block to aggregate the spatial information. Specifically, we uniformly take the central agent i 's latest trajectory segment $\mathbf{p}_i^T - \mathbf{p}_i^{T-1}$ as the reference vector of the local region and rotate all local vectors according to the reference vector's orientation θ_i . Based on the rotated vectors and their associated semantic attributes, we use Multi-Layer Perceptrons (MLPs) to obtain the central agent i 's embedding $\mathbf{z}_i^t \in \mathbb{R}^{d_h}$ and any neighboring agent j 's embedding $\mathbf{z}_{ij}^t \in \mathbb{R}^{d_h}$ at any time step t :

$$\mathbf{z}_i^t = \phi_{\text{center}}([\mathbf{R}_i^\top (\mathbf{p}_i^t - \mathbf{p}_i^{t-1}), \mathbf{a}_i]), \quad (1)$$

$$\mathbf{z}_{ij}^t = \phi_{\text{nbr}}([\mathbf{R}_i^\top (\mathbf{p}_j^t - \mathbf{p}_j^{t-1}), \mathbf{R}_i^\top (\mathbf{p}_j^t - \mathbf{p}_i^t), \mathbf{a}_j]), \quad (2)$$

where $\phi_{\text{center}}(\cdot)$ and $\phi_{\text{nbr}}(\cdot)$ are two different MLP blocks, $\mathbf{R}_i \in \mathbb{R}^{2 \times 2}$ is the rotation matrix parameterized by θ_i , $\mathbf{a}_i$

and $\mathbf{a}_j$ are the semantic attributes of agent i and agent j , respectively. Since all geometric attributes are normalized w.r.t. the central agent before they are fed into MLPs, these embeddings are unaffected by the rotation of the global coordinate frame. Apart from the trajectory segments, the inputs of $\phi_{\text{nbr}}(\cdot)$ also contain neighboring agents' position vectors relative to the central agent, making the neighboring embeddings spatially aware. The embedding of the central agent is then converted to the query vector, and the embeddings of the neighboring agents are used to calculate the key and the value vectors:

$$\mathbf{q}_i^t = \mathbf{W}^{Q^{\text{space}}} \mathbf{z}_i^t, \quad \mathbf{k}_{ij}^t = \mathbf{W}^{K^{\text{space}}} \mathbf{z}_{ij}^t, \quad \mathbf{v}_{ij}^t = \mathbf{W}^{V^{\text{space}}} \mathbf{z}_{ij}^t, \quad (3)$$

where $\mathbf{W}^{Q^{\text{space}}}, \mathbf{W}^{K^{\text{space}}}, \mathbf{W}^{V^{\text{space}}} \in \mathbb{R}^{d_k \times d_h}$ are learnable matrices for linear projection and d_k is the dimension of the transformed vectors. The resulting query, key, and value vectors are taken as inputs to the scaled dot-product attention block:

$$\alpha_i^t = \text{softmax} \left(\frac{\mathbf{q}_i^{t\top}}{\sqrt{d_k}} \cdot [\{\mathbf{k}_{ij}^t\}_{j \in \mathcal{N}_i}] \right), \quad (4)$$

$$\mathbf{m}_i^t = \sum_{j \in \mathcal{N}_i} \alpha_{ij}^t \mathbf{v}_{ij}^t, \quad (5)$$

$$\mathbf{g}_i^t = \text{sigmoid}(\mathbf{W}^{\text{gate}} [\mathbf{z}_i^t, \mathbf{m}_i^t]), \quad (6)$$

$$\hat{\mathbf{z}}_i^t = \mathbf{g}_i^t \odot \mathbf{W}^{\text{self}} \mathbf{z}_i^t + (1 - \mathbf{g}_i^t) \odot \mathbf{m}_i^t, \quad (7)$$

where $\mathcal{N}_i$ is the set of agent i 's neighbors, $\mathbf{W}^{\text{gate}}$ and $\mathbf{W}^{\text{self}}$ are learnable matrices, and $\odot$ denotes element-wise product. Compared with the standard scaled dot-product attention [46], our variant uses a gating function to fuse the environmental features $\mathbf{m}_i^t$ with the central agent's features $\mathbf{z}_i^t$, enabling the block to have more control over the feature update. Like the original Transformer architecture, our attention block can also be extended to multiple heads. The outputs of the multi-head attention block are passed through an MLP block to obtain the spatial embedding $\mathbf{s}_i^t \in \mathbb{R}^{d_h}$ of agent i at time step t . In addition, we apply Layer Normalization [3] before each block and residual connections [24] after each block. In practice, this module can be implemented using efficient scatter and gather operations to parallelize the learning across all local regions and all time steps.

Temporal Dependency. To further capture the temporal information of each local region, we employ a temporal Transformer encoder on top of the agent-agent interaction module. For any central agent i , the input sequence of this module is composed of the embeddings $\{\mathbf{s}_i^t\}_{t=1}^T$ returned by the agent-agent interaction module at different time steps. Similar to BERT [14], we append an extra learnable token $\mathbf{s}^{T+1} \in \mathbb{R}^{d_h}$ to the end of the input sequence. Then, we add learnable positional embeddings to all tokens and stack the

We adopt the negative log-likelihood as the regression loss:

$$\mathcal{L}_{\text{reg}} = -\frac{1}{NH} \sum_{i=1}^N \sum_{t=T+1}^{T+H} \log P \left(\mathbf{R}_i^\top (\mathbf{p}_i^t - \mathbf{p}_i^T) \mid \hat{\boldsymbol{\mu}}_i^t, \hat{\mathbf{b}}_i^t \right), \quad (15)$$

where $P(\cdot \mid \cdot)$ is the probability density function of Laplace distribution, and $\{\hat{\boldsymbol{\mu}}_i^t\}_{t=T+1}^{T+H}$, $\{\hat{\mathbf{b}}_i^t\}_{t=T+1}^{T+H}$ are the locations and the uncertainties of the best-predicted trajectory for agent i . We use the cross-entropy loss as the classification loss to optimize the mixing coefficients.

4. Experiments

4.1. Experimental Setup

Dataset. We evaluate our prediction framework on the large-scale Argoverse motion forecasting dataset [10], which provides the trajectories of agents and the high-definition map data. The dataset contains 323557 real-world driving scenarios and is split into training, validation, and test sets, with 205942, 39472, and 78143 samples. All training and validation scenarios are 5-second sequences sampled at 10 Hz, while only the first 2-second trajectories are publicly available in the test set. Given the initial 2-second observations, the Argoverse Motion Forecasting Challenge requires predicting the 3-second future movements of the agents.

Metrics. We evaluate our model on the standard metrics for motion prediction, including minimum Average Displacement Error (minADE), minimum Final Displacement Error (minFDE), and Miss Rate (MR). These metrics allow models to forecast up to 6 trajectories for each agent. The metric minADE measures the ℓ_2 distance in meters between the best-predicted trajectory and the ground-truth trajectory averaged over all future time steps, while minFDE measures the error at the final future time step. The best-predicted trajectory is defined as the one that has the minimum endpoint error. MR refers to the ratio of scenarios where the distance between the ground-truth endpoint and the best-predicted endpoint is above 2.0 meters.

Implementation Details. We train our model for 64 epochs on an RTX 2080 Ti GPU using AdamW optimizer [35], with the batch size, initial learning rate, weight decay, and dropout rate set to 32, 3×10^{-4} , 1×10^{-4} , and 0.1, respectively. The learning rate is decayed using the cosine annealing scheduler [34]. Our model consists of 1 layer of agent-agent and agent-lane interaction module, 4 layers of temporal learning module, and 3 layers of global interaction module. The number of heads in all multi-head attention blocks is 8. The radius of all local regions is 50 meters. We follow the convention in the baselines and set the number of the predicted modes F to 6. We do not use tricks such as ensemble methods and data augmentation. We conduct experiments based on a small model with 64 hidden

A-A	Temporal	A-L	Global	minADE	minFDE	MR
	✓	✓	✓	0.71	1.07	0.11
✓		✓	✓	1.00	1.56	0.21
✓	✓		✓	0.77	1.25	0.14
✓	✓	✓		0.73	1.13	0.12
✓	✓	✓	✓	0.69	1.04	0.10

Table 1. Ablation studies on the components of our framework.

Gated Update	Temporal Mask	minADE	minFDE	MR
		0.70	1.07	0.11
	✓	0.70	1.05	0.11
✓		0.70	1.05	0.11
✓	✓	0.69	1.04	0.10

Table 2. Ablation studies on the attention blocks.

units and a large model with 128 hidden units, termed as HiVT-64 and HiVT-128, respectively.

4.2. Ablation Studies

We conduct ablation studies on the Argoverse validation set. Unless specified, experimental results are based on our 64-dimension model HiVT-64.

Importance of Each Module. We demonstrate each module’s contribution to the prediction performance by alternately removing one of the components. As shown in Tab. 1, each component can improve the performance to a certain degree. First, without the agent-agent interaction module, the model cannot capture fine-grained local interactions at previous time steps and suffers from performance drop. We also note that adding more layers of this module can further improve the performance, but we keep using one layer for higher efficiency. Second, the temporal learning module has the most significant impact on the performance since inferring the future motions of traffic agents in highly dynamic traffic scenarios relies heavily on the historical information. Third, the lane information plays a crucial role in motion predictions since traffic agents usually move along the lanes due to the constraint of traffic rules. Moreover, the global interaction module can noticeably improve the prediction performance. This result validates its capacity for capturing long-range dependencies.

Ablation Studies on the Attention Blocks. We evaluate the effect of the gated update function in the spatial attention block and the temporal mask in the temporal attention block. As shown in Tab. 2, using the gating function can improve the prediction performance, presumably because some agents do not interact much with the environment. The results in Tab. 2 also show that removing the temporal mask in the temporal attention block can lead to worse performance, which suggests that preventing tokens from attending to subsequent time steps is beneficial to the model.